JavaScript is disabled on your browser.



compact1, compact2, compact3

java.text

# Class DateFormatSymbols

java.lang.Object
    java.text.DateFormatSymbols

**All Implemented Interfaces:**

`Serializable`, `Cloneable`

---

```
public class DateFormatSymbols
extends Object
implements Serializable, Cloneable
```

`DateFormatSymbols` is a public class for encapsulating localizable date-time formatting data, such as the names of the months, the names of the days of the week, and the time zone data. `SimpleDateFormat` uses `DateFormatSymbols` to encapsulate this information.

Typically you shouldn't use `DateFormatSymbols` directly. Rather, you are encouraged to create a date-time formatter with the `DateFormat` class's factory methods: `getTimeInstance`, `getDateInstance`, or `getDateTimeInstance`. These methods automatically create a `DateFormatSymbols` for the formatter so that you don't have to. After the formatter is created, you may modify its format pattern using the `setPattern` method. For more information about creating formatters using `DateFormat`'s factory methods, see `DateFormat`.

If you decide to create a date-time formatter with a specific format pattern for a specific locale, you can do so with:

```
new SimpleDateFormat(aPattern, DateFormatSymbols.getInstance(aLocale)).
```

`DateFormatSymbols` objects are cloneable. When you obtain a `DateFormatSymbols` object, feel free to modify the date-time formatting data. For instance, you can replace the localized date-time format pattern characters with the ones that you

feel easy to remember. Or you can change the representative cities to your favorite ones.

New `DateFormatSymbols` subclasses may be added to support `SimpleDateFormat` for date-time formatting for additional locales.

**See Also:**

`DateFormat`, `SimpleDateFormat`, `SimpleTimeZone`, `Serialized Form`

## *Constructor Summary*

**Constructors**

| Constructor | Description |
|---|---|
| **`DateFormatSymbols`**`()` | Construct a DateFormatSymbols object by loading format data from resources for the default **`FORMAT`** locale. |
| **`DateFormatSymbols`**`(`**`Locale`**` locale)` | Construct a DateFormatSymbols object by loading format data from resources for the given locale. |

## *Method Summary*

**Concrete Methods**

| Modifier and Type | Method | Descr |
|---|---|---|
| **`Object`** | **`clone`**`()` | Over Clone |
| `boolean` | **`equals`**`(`**`Object`**` obj)` | Over |
| **`String`**`[]` | **`getAmPmStrings`**`()` | Gets |
| `static` **`Locale`**`[]` | **`getAvailableLocales`**`()` | Retu all lo the g meth class locali |
| **`String`**`[]` | **`getEras`**`()` | Gets |
| `static` **`DateFormatSymbols`** | **`getInstance`**`()` | Gets Date |

| | | |
|---|---|---|
| | | insta<br>defau |
| static **DateFormatSymbols** | **getInstance**(**Locale** locale) | Gets<br>Date<br>insta<br>speci |
| **String** | **getLocalPatternChars**() | Gets<br>time<br>chara |
| **String**[] | **getMonths**() | Gets |
| **String**[] | **getShortMonths**() | Gets<br>string |
| **String**[] | **getShortWeekdays**() | Gets<br>string |
| **String**[] | **getWeekdays**() | Gets<br>string |
| **String**[][] | **getZoneStrings**() | Gets<br>string |
| int | **hashCode**() | Over |
| void | **setAmPmStrings**(**String**[] newAmpms) | Sets |
| void | **setEras**(**String**[] newEras) | Sets |
| void | **setLocalPatternChars**(**String** newLocalPatternChars) | Sets<br>time<br>chara |
| void | **setMonths**(**String**[] newMonths) | Sets |
| void | **setShortMonths**(**String**[] newShortMonths) | Sets<br>string |
| void | **setShortWeekdays**(**String**[] newShortWeekdays) | Sets<br>string |
| void | **setWeekdays**(**String**[] newWeekdays) | Sets<br>string |
| void | **setZoneStrings**(**String**[][] newZoneStrings) | Sets<br>string |

### Methods inherited from class java.lang.Object

finalize, getClass, notify, notifyAll, toString, wait, wait, wait

## *Constructor Detail*

### DateFormatSymbols

```
public DateFormatSymbols()
```

Construct a DateFormatSymbols object by loading format data from resources for the default `FORMAT` locale. This constructor can only construct instances for the locales supported by the Java runtime environment, not for those supported by installed `DateFormatSymbolsProvider` implementations. For full locale coverage, use the `getInstance` method.

This is equivalent to calling `DateFormatSymbols(Locale.getDefault(Locale.Category.FORMAT))`.

**Throws:**

`MissingResourceException` - if the resources for the default locale cannot be found or cannot be loaded.

**See Also:**

`getInstance()`, `Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

### DateFormatSymbols

```
public DateFormatSymbols(Locale locale)
```

Construct a DateFormatSymbols object by loading format data from resources for the given locale. This constructor can only construct instances for the locales supported by the Java runtime environment, not for those supported by installed `DateFormatSymbolsProvider` implementations. For full locale coverage, use the `getInstance` method.

**Parameters:**

`locale` - the desired locale

**Throws:**

`MissingResourceException` - if the resources for the specified locale cannot be found or cannot be loaded.

**See Also:**

`getInstance(Locale)`

## *Method Detail*

### getAvailableLocales

```
public static Locale[] getAvailableLocales()
```

Returns an array of all locales for which the `getInstance` methods of this class can return localized instances. The returned array represents the union of locales supported by the Java runtime and by installed `DateFormatSymbolsProvider` implementations. It must contain at least a `Locale` instance equal to `Locale.US`.

**Returns:**

An array of locales for which localized DateFormatSymbols instances are available.

**Since:**

1.6

### getInstance

```
public static final DateFormatSymbols getInstance()
```

Gets the `DateFormatSymbols` instance for the default locale. This method provides access to `DateFormatSymbols` instances for locales supported by the Java runtime itself as well as for those supported by installed `DateFormatSymbolsProvider` implementations.

This is equivalent to calling `getInstance(Locale.getDefault(Locale.Category.FORMAT))`.

**Returns:**

a DateFormatSymbols instance.

**Since:**

1.6

**See Also:**

`Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

### getInstance

```
public static final DateFormatSymbols getInstance(Locale locale)
```

Gets the `DateFormatSymbols` instance for the specified locale. This method provides access to `DateFormatSymbols` instances for locales supported by the

Java runtime itself as well as for those supported by installed `DateFormatSymbolsProvider` implementations.

**Parameters:**

`locale - the given locale.`

**Returns:**

`a DateFormatSymbols instance.`

**Throws:**

`NullPointerException - if locale is null`

**Since:**

`1.6`

### getEras

```
public String[] getEras()
```

Gets era strings. For example: "AD" and "BC".

**Returns:**

`the era strings.`

### setEras

```
public void setEras(String[] newEras)
```

Sets era strings. For example: "AD" and "BC".

**Parameters:**

`newEras - the new era strings.`

### getMonths

```
public String[] getMonths()
```

Gets month strings. For example: "January", "February", etc.

If the language requires different forms for formatting and stand-alone usages, this method returns month names in the formatting form. For example, the preferred month name for January in the Czech language is *ledna* in the formatting form, while it is *leden* in the stand-alone form. This method returns `"ledna"` in this case. Refer to the Calendar Elements in the Unicode Locale Data Markup Language (LDML) specification for more details.

**Returns:**

the month strings.

### setMonths

```
public void setMonths(String[] newMonths)
```

Sets month strings. For example: "January", "February", etc.

**Parameters:**

`newMonths` - the new month strings.

### getShortMonths

```
public String[] getShortMonths()
```

Gets short month strings. For example: "Jan", "Feb", etc.

If the language requires different forms for formatting and stand-alone usages, This method returns short month names in the formatting form. For example, the preferred abbreviation for January in the Catalan language is *de gen.* in the formatting form, while it is *gen.* in the stand-alone form. This method returns `"de gen."` in this case. Refer to the Calendar Elements in the Unicode Locale Data Markup Language (LDML) specification for more details.

**Returns:**

the short month strings.

### setShortMonths

```
public void setShortMonths(String[] newShortMonths)
```

Sets short month strings. For example: "Jan", "Feb", etc.

**Parameters:**

`newShortMonths` - the new short month strings.

### getWeekdays

```
public String[] getWeekdays()
```

Gets weekday strings. For example: "Sunday", "Monday", etc.

**Returns:**

the weekday strings. Use Calendar.SUNDAY, Calendar.MONDAY, etc. to index the result array.

### setWeekdays

```
public void setWeekdays(String[] newWeekdays)
```

Sets weekday strings. For example: "Sunday", "Monday", etc.

**Parameters:**

newWeekdays - the new weekday strings. The array should be indexed by Calendar.SUNDAY, Calendar.MONDAY, etc.

### getShortWeekdays

```
public String[] getShortWeekdays()
```

Gets short weekday strings. For example: "Sun", "Mon", etc.

**Returns:**

the short weekday strings. Use Calendar.SUNDAY, Calendar.MONDAY, etc. to index the result array.

### setShortWeekdays

```
public void setShortWeekdays(String[] newShortWeekdays)
```

Sets short weekday strings. For example: "Sun", "Mon", etc.

**Parameters:**

newShortWeekdays - the new short weekday strings. The array should be indexed by Calendar.SUNDAY, Calendar.MONDAY, etc.

### getAmPmStrings

```
public String[] getAmPmStrings()
```

Gets ampm strings. For example: "AM" and "PM".

**Returns:**

the ampm strings.

### setAmPmStrings

```
public void setAmPmStrings(String[] newAmpms)
```

Sets ampm strings. For example: "AM" and "PM".

**Parameters:**

`newAmpms` - the new ampm strings.

### getZoneStrings

```
public String[][] getZoneStrings()
```

Gets time zone strings. Use of this method is discouraged; use `TimeZone.getDisplayName()` instead.

The value returned is a two-dimensional array of strings of size *n* by *m*, where *m* is at least 5. Each of the *n* rows is an entry containing the localized names for a single `TimeZone`. Each such row contains (with `i` ranging from 0..*n*-1):

- `zoneStrings[i][0]` - time zone ID
- `zoneStrings[i][1]` - long name of zone in standard time
- `zoneStrings[i][2]` - short name of zone in standard time
- `zoneStrings[i][3]` - long name of zone in daylight saving time
- `zoneStrings[i][4]` - short name of zone in daylight saving time

The zone ID is *not* localized; it's one of the valid IDs of the `TimeZone` class that are not custom IDs. All other entries are localized names. If a zone does not implement daylight saving time, the daylight saving time names should not be used.

If `setZoneStrings` has been called on this `DateFormatSymbols` instance, then the strings provided by that call are returned. Otherwise, the returned array contains names provided by the Java runtime and by installed `TimeZoneNameProvider` implementations.

**Returns:**

the time zone strings.

**See Also:**

`setZoneStrings(String[][])`

### setZoneStrings

```
public void setZoneStrings(String[][] newZoneStrings)
```

Sets time zone strings. The argument must be a two-dimensional array of strings of size *n* by *m*, where *m* is at least 5. Each of the *n* rows is an entry containing the localized names for a single `TimeZone`. Each such row contains (with `i` ranging from 0..*n*-1):

- `zoneStrings[i][0]` - time zone ID
- `zoneStrings[i][1]` - long name of zone in standard time
- `zoneStrings[i][2]` - short name of zone in standard time
- `zoneStrings[i][3]` - long name of zone in daylight saving time
- `zoneStrings[i][4]` - short name of zone in daylight saving time

The zone ID is *not* localized; it's one of the valid IDs of the `TimeZone` class that are not custom IDs. All other entries are localized names.

**Parameters:**

`newZoneStrings - the new time zone strings.`

**Throws:**

`IllegalArgumentException - if the length of any row in newZoneStrings is less than 5`

`NullPointerException - if newZoneStrings is null`

**See Also:**

`getZoneStrings()`

### getLocalPatternChars

```
public String getLocalPatternChars()
```

Gets localized date-time pattern characters. For example: 'u', 't', etc.

**Returns:**

`the localized date-time pattern characters.`

### setLocalPatternChars

```
public void setLocalPatternChars(String newLocalPatternChars)
```

Sets localized date-time pattern characters. For example: 'u', 't', etc.

**Parameters:**

`newLocalPatternChars - the new localized date-time pattern characters.`

### clone

```
public Object clone()
```

Overrides Cloneable

**Overrides:**

`clone` in class `Object`

**Returns:**

a clone of this instance.

**See Also:**

`Cloneable`

### hashCode

```
public int hashCode()
```

Override hashCode. Generates a hash code for the DateFormatSymbols object.

**Overrides:**

`hashCode` in class `Object`

**Returns:**

a hash code value for this object.

**See Also:**

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

### equals

```
public boolean equals(Object obj)
```

Override equals

**Overrides:**

`equals` in class `Object`

**Parameters:**

`obj - the reference object with which to compare.`

**Returns:**

`true if this object is the same as the obj argument; false otherwise.`

**See Also:**

`Object.hashCode()`, `HashMap`




Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.